

Algorithmique

Récurtivité et tris

Cédric Lhoussaine

Université de Lille / IUT de Lille

Objectifs de ce cours

- ✓ Comprendre les principes du **tri fusion** et du **quicksort**.
- ✓ Établir et résoudre les **équations de récurrence** pour chaque algorithme.
- ✓ Comparer les **complexités** (pire cas, meilleur cas, cas moyen).

Introduction et rappels

- Une fonction est **récursive** si elle s'appelle elle-même

- Une fonction est **récursive** si elle s'appelle elle-même
- **Composants clés:**
 1. *Cas de base* (condition d'arrêt).
 2. *Appel récursif* (problème plus petit).
 3. *Progression* vers le cas de base.

- Une fonction est **récursive** si elle s'appelle elle-même
- **Composants clés:**
 1. *Cas de base* (condition d'arrêt).
 2. *Appel récursif* (problème plus petit).
 3. *Progression* vers le cas de base.
- **Exemples vus précédemment:**
 - Factorielle : $T(n) = T(n - 1) + c \rightarrow \Theta(n)$
 - Fibonacci : $T(n) = T(n - 1) + T(n - 2) + c \rightarrow \Theta(2^n)$

Un **modèle récursif** pour résoudre des problèmes:

1. **Diviser** (décomposer) le problème en **sous-problèmes** plus petits
2. **Résoudre** récursivement les sous-problèmes
3. **Combiner** les solutions des sous-problèmes.

Tri fusion





Inventé par John von Neumann en 1945

J. Von Neumann (1903-1957)



J. Von Neumann (1903-1957)

Inventé par John von Neumann en 1945

Idée principale:

- **Diviser** le tableau en 2 moitiés
- **Trier récursivement** chaque moitié
- **Combiner**, en les fusionnant, les 2 moitiés triées.

Propriétés

- **Stable**: préserve l'ordre des éléments égaux
- **Complexité** : $\Theta(n \log(n))$ (même dans le pire cas).

Tri fusion: principe général

1. Diviser :

- Si le tableau a **0 ou 1 élément**, il est déjà trié. (*cas terminal*)
- Sinon, le diviser en **2 sous-tableaux** de taille $\lfloor \frac{n}{2} \rfloor$ et $\lceil \frac{n}{2} \rceil$.

2. Résoudre:

- Trier récursivement chaque sous-tableau.

3. Combiner :

- Fusionner les 2 sous-tableaux triés en un seul tableau trié.

(*exemple...*)

Fusion de deux tableaux triés

- **Entrées :**

Tableau A avec sous-tableaux triés $A[p...q]$ et $A[q + 1...r]$.

- **Sortie :**

Tableau $A[p..r]$ trié.

- **Idée :**

- comparer les éléments des 2 sous-tableaux *un par un et de gauche à droite*.
- placer le plus petit dans le tableau final.

(*exemple...*)

Algorithme de fusion

```
1: function FUSION( $A, p, q, r$ )
2:    $n_1, n_2 \leftarrow q - p + 1, r - q$ 
3:   for  $j \leftarrow 0$  to  $n_1 - 1$  do
4:      $L[j] \leftarrow A[p + j]$ 
5:   end
6:   for  $j \leftarrow 0$  to  $n_2 - 1$  do
7:      $R[j] \leftarrow A[q + j + 1]$ 
8:   end
9:    $L[n_1], R[n_2] \leftarrow \infty, \infty$ 
10:   $i, j \leftarrow 0, 0$ 
11:  for  $k \leftarrow p$  to  $r$  do
12:    if  $L[i] \leq R[j]$  then
13:       $A[k] \leftarrow L[i++]$ 
14:    else
15:       $A[k] \leftarrow R[j++]$ 
16:    end
17:  end
18: end
```

▷ Création de tableaux temporaires $L[0\dots n_1]$ et $R[0\dots n_2]$

▷ Sentinelles pour simplifier la fusion

Coût de la fusion

```
1: function FUSION( $A, p, q, r$ )
2:    $n_1, n_2 \leftarrow q - p + 1, r - q$   $\triangleright c_1$ 
3:   for  $j \leftarrow 0$  to  $n_1 - 1$  do  $\triangleright n_1 \times c_2$ 
4:      $L[j] \leftarrow A[p + i]$   $\triangleright (n_1 - 1) \times c_3$ 
5:   end
6:   for  $j \leftarrow 0$  to  $n_2 - 1$  do  $\triangleright n_2 \times c_4$ 
7:      $R[j] \leftarrow A[q + j + 1]$   $\triangleright (n_2 - 1) \times c_5$ 
8:   end
9:    $L[n_1], R[n_2] \leftarrow \infty, \infty$   $\triangleright c_6$ 
10:   $i, j \leftarrow 0, 0$   $\triangleright c_7$ 
11:  for  $k \leftarrow p$  to  $r$  do  $\triangleright (n_1 + n_2 + 1) \times c_8$ 
12:    if  $L[i] \leq R[j]$  then  $\triangleright (n_1 + n_2)c_9$ 
13:       $A[k] \leftarrow L[i++]$   $\triangleright m_1 \times c_{10}$ 
14:    else
15:       $A[k] \leftarrow R[j++]$   $\triangleright m_2 \times c_{10}$ 
16:    end
17:  end
18: end
```

où $n = n_1 + n_2$ et $m_1 + m_2 = n_1 + n_2 - 1 = n - 1$.

$$T_{\text{fusion}}(n) = \Theta(n)$$

(au tableau...)

Algorithme de tri fusion

```
1: function TRI-FUSION( $A, p, r$ )
2:   if  $p < r$  then
3:      $q \leftarrow \lfloor \frac{p+r}{2} \rfloor$ 
4:     TRI-FUSION( $A, p, q$ )
5:     TRI-FUSION( $A, q + 1, r$ )
6:     FUSION( $A, p, q, r$ )
7:   end
8: end
```

- ▷ Diviser
- ▷ Résoudre (moitié gauche)
- ▷ Résoudre (moitié droite)
- ▷ Combiner

Équation de récurrence du coût du tri fusion

```
1: function TRI-FUSION( $A, p, r$ )
2:   if  $p < r$  then
3:      $q \leftarrow \lfloor \frac{p+r}{2} \rfloor$ 
4:     TRI-FUSION( $A, p, q$ )
5:     TRI-FUSION( $A, q + 1, r$ )
6:     FUSION( $A, p, q, r$ )
7:   end
8: end
```

- $T(n)$ = temps pour trier un tableau de taille n .
- On suppose n est une puissance de 2 (pour simplifier).

Équation de récurrence du coût du tri fusion

```
1: function TRI-FUSION( $A, p, r$ )
2:   if  $p < r$  then
3:      $q \leftarrow \lfloor \frac{p+r}{2} \rfloor$ 
4:     TRI-FUSION( $A, p, q$ )
5:     TRI-FUSION( $A, q + 1, r$ )
6:     FUSION( $A, p, q, r$ )
7:   end
8: end
```

- $T(n)$ = temps pour trier un tableau de taille n .
- On suppose n est une puissance de 2 (pour simplifier).

Équation de récurrence :

- Cas de base:

Équation de récurrence du coût du tri fusion

```
1: function TRI-FUSION( $A, p, r$ )
2:   if  $p < r$  then
3:      $q \leftarrow \lfloor \frac{p+r}{2} \rfloor$ 
4:     TRI-FUSION( $A, p, q$ )
5:     TRI-FUSION( $A, q + 1, r$ )
6:     FUSION( $A, p, q, r$ )
7:   end
8: end
```

- $T(n)$ = temps pour trier un tableau de taille n .
- On suppose n est une puissance de 2 (pour simplifier).

Équation de récurrence :

- Cas de base: quand $p \geq r$ ($n = 0$ ou 1)
- Cas récursif ($n > 1$):
 - Diviser:

Équation de récurrence du coût du tri fusion

```
1: function TRI-FUSION( $A, p, r$ )
2:   if  $p < r$  then
3:      $q \leftarrow \lfloor \frac{p+r}{2} \rfloor$ 
4:     TRI-FUSION( $A, p, q$ )
5:     TRI-FUSION( $A, q + 1, r$ )
6:     FUSION( $A, p, q, r$ )
7:   end
8: end
```

- $T(n)$ = temps pour trier un tableau de taille n .
- On suppose n est une puissance de 2 (pour simplifier).

Équation de récurrence :

- Cas de base: quand $p \geq r$ ($n = 0$ ou 1)
- Cas récursif ($n > 1$):
 - Diviser: $q \leftarrow \frac{p+r}{2}$
 - Résoudre :

Équation de récurrence du coût du tri fusion

```
1: function TRI-FUSION( $A, p, r$ )
2:   if  $p < r$  then
3:      $q \leftarrow \lfloor \frac{p+r}{2} \rfloor$ 
4:     TRI-FUSION( $A, p, q$ )
5:     TRI-FUSION( $A, q + 1, r$ )
6:     FUSION( $A, p, q, r$ )
7:   end
8: end
```

- $T(n)$ = temps pour trier un tableau de taille n .
- On suppose n est une puissance de 2 (pour simplifier).

Équation de récurrence :

- Cas de base: quand $p \geq r$ ($n = 0$ ou 1)
- Cas récursif ($n > 1$):
 - Diviser: $q \leftarrow \frac{p+r}{2}$
 - Résoudre : 2 appels récursifs de tableaux de taille $\frac{n}{2}$
 - Combiner :

Équation de récurrence du coût du tri fusion

```
1: function TRI-FUSION( $A, p, r$ )
2:   if  $p < r$  then
3:      $q \leftarrow \lfloor \frac{p+r}{2} \rfloor$ 
4:     TRI-FUSION( $A, p, q$ )
5:     TRI-FUSION( $A, q + 1, r$ )
6:     FUSION( $A, p, q, r$ )
7:   end
8: end
```

- $T(n)$ = temps pour trier un tableau de taille n .
- On suppose n est une puissance de 2 (pour simplifier).

Équation de récurrence :

- Cas de base: quand $p \geq r$ ($n = 0$ ou 1)
- Cas récursif ($n > 1$):
 - Diviser: $q \leftarrow \frac{p+r}{2}$
 - Résoudre : 2 appels récursifs de tableaux de taille $\frac{n}{2}$
 - Combiner : fusion de 2 tableaux de taille $\frac{n}{2}$

En approximant le coût de la fusion à une fonction linéaire $T_{\text{fusion}}(n) = an + b$,

$$T(n) = \begin{cases} c & \text{si } n \leq 1 \\ 2T(\frac{n}{2}) + an + b & \text{sinon} \end{cases}$$

En approximant le coût de la fusion à une fonction linéaire $T_{\text{fusion}}(n) = an + b$,

$$T(n) = \begin{cases} c & \text{si } n \leq 1 \\ 2T(\frac{n}{2}) + an + b & \text{sinon} \end{cases}$$

(résolution au tableau...)

En approximant le coût de la fusion à une fonction linéaire $T_{\text{fusion}}(n) = an + b$,

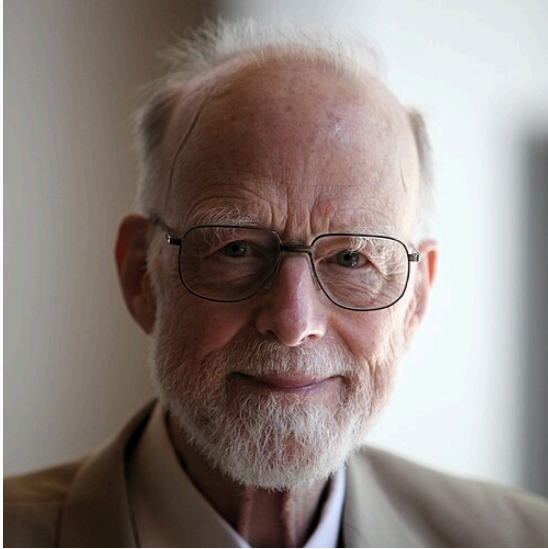
$$T(n) = \begin{cases} c & \text{si } n \leq 1 \\ 2T(\frac{n}{2}) + an + b & \text{sinon} \end{cases}$$

(résolution au tableau...)

$$T(n) = \Theta(n \log(n))$$

Tri rapide (Quicksort)





Inventé par Tony Hoare (1961)

T. Hoare (1934 - 2026)

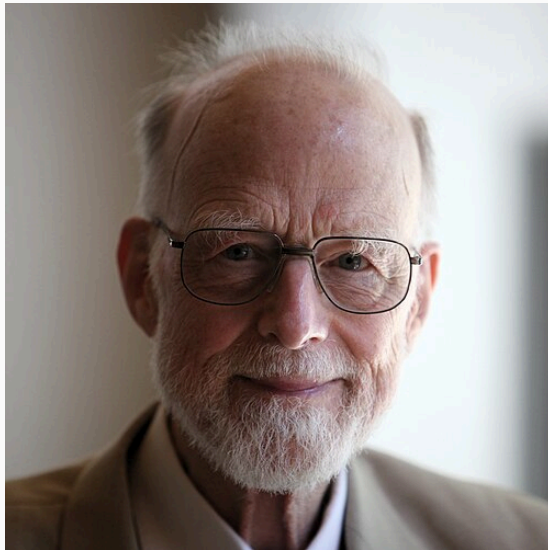


T. Hoare (1934 - 2026)

Inventé par Tony Hoare (1961)

Propriétés

- Instable (l'ordre des éléments égaux peut changer).
- algorithme « en place » (ne nécessite pas de mémoire additionnelle)



T. Hoare (1934 - 2026)

Inventé par Tony Hoare (1961)

Propriétés

- Instable (l'ordre des éléments égaux peut changer).
- algorithme « en place » (ne nécessite pas de mémoire additionnelle)

Complexité :

- Meilleur cas : $\Theta(n \log(n))$
- Pire cas : $\Theta(n^2)$ (si le pivot est mal choisi)
- Cas moyen : $\Theta(n \log(n))$

1. **Choisir un pivot** (par exemple : le dernier élément).
2. **Partitionner** le tableau, i.e. réorganiser le tableau pour que:
 - tous les éléments $<$ pivot soient à gauche du pivot;
 - tous les éléments $\geq$ pivot soient à droite du pivot;

Le pivot est alors *à sa position finale*.

3. **Trier récursivement** les sous-tableaux à gauche et à droite du pivot.

(exemple & Wooclap...)

```
1: function PARTITION( $A, p, r$ )
2:    $x \leftarrow A[r]$ 
3:    $i \leftarrow p - 1$ 
4:   for  $j \leftarrow p$  to  $r - 1$  do
5:     if  $A[j] \leq x$  then
6:        $i \leftarrow i + 1$ 
7:       PERMUTER( $A[i], A[j]$ )
8:     end
9:   end
10:  PERMUTER( $A[i + 1], A[r]$ )
11:  return  $i + 1$ 
12: end
```

(exemple...)

```
1: function PARTITION( $A, p, r$ )
2:    $x \leftarrow A[r]$ 
3:    $i \leftarrow p - 1$ 
4:   for  $j \leftarrow p$  to  $r - 1$  do
5:     if  $A[j] \leq x$  then
6:        $i \leftarrow i + 1$ 
7:       PERMUTER( $A[i], A[j]$ )
8:     end
9:   end
10:  PERMUTER( $A[i + 1], A[r]$ )
11:  return  $i + 1$ 
12: end
```

(exemple...)

Complexité: On parcourt toujours tous les éléments de p à r , donc

$$T_{\text{partition}}(n) = \Theta(n)$$

où $n = r - p$

Quicksort

```
1: function QUICKSORT( $A, p, r$ )
2:   if  $p < r$  then
3:      $q \leftarrow \text{PARTITION}(A, p, r)$ 
4:     QUICKSORT( $A, p, q - 1$ )
5:     QUICKSORT( $A, q + 1, r$ )
6:   end
7: end
```

Appel initial: $\text{QUICKSORT}(A, 0, \text{longueur}(A) - 1)$

Coût du Quicksort dans le cas le plus défavorable

Dans ce cas, **le partitionnement est le plus déséquilibré**, i.e. il produit toujours un tableau de taille 0 et un tableau de taille maximale (ie. $n - 1$)

Dans ce cas, **le partitionnement est le plus déséquilibré**, i.e. il produit toujours un tableau de taille 0 et un tableau de taille maximale (ie. $n - 1$)

- le partitionnement est toujours linéaire (de la forme $an + b$)

Coût du Quicksort dans le cas le plus défavorable

Dans ce cas, **le partitionnement est le plus déséquilibré**, i.e. il produit toujours un tableau de taille 0 et un tableau de taille maximale (ie. $n - 1$)

- le partitionnement est toujours linéaire (de la forme $an + b$)
- le 1er appel récursif sur un tableau vide est constant (c)

Dans ce cas, **le partitionnement est le plus déséquilibré**, i.e. il produit toujours un tableau de taille 0 et un tableau de taille maximale (ie. $n - 1$)

- le partitionnement est toujours linéaire (de la forme $an + b$)
- le 1er appel récursif sur un tableau vide est constant (c)
- le 2ème appel récursif sur un tableau vide de taille $n - 1$ est $T^{\max}(n - 1)$

Coût du Quicksort dans le cas le plus défavorable

Dans ce cas, **le partitionnement est le plus déséquilibré**, i.e. il produit toujours un tableau de taille 0 et un tableau de taille maximale (ie. $n - 1$)

- le partitionnement est toujours linéaire (de la forme $an + b$)
- le 1er appel récursif sur un tableau vide est constant (c)
- le 2ème appel récursif sur un tableau vide de taille $n - 1$ est $T^{\max}(n - 1)$

d'où l'équation de récurrence:

$$T^{\max}(n) = \begin{cases} c & \text{si } n = 0 \\ T^{\max}(n - 1) + an + b & \text{sinon} \end{cases}$$

Qui a pour solution

Coût du Quicksort dans le cas le plus défavorable

Dans ce cas, **le partitionnement est le plus déséquilibré**, i.e. il produit toujours un tableau de taille 0 et un tableau de taille maximale (ie. $n - 1$)

- le partitionnement est toujours linéaire (de la forme $an + b$)
- le 1er appel récursif sur un tableau vide est constant (c)
- le 2ème appel récursif sur un tableau vide de taille $n - 1$ est $T^{\max}(n - 1)$

d'où l'équation de récurrence:

$$T^{\max}(n) = \begin{cases} c & \text{si } n = 0 \\ T^{\max}(n - 1) + an + b & \text{sinon} \end{cases}$$

Qui a pour solution $T^{\max}(n) = c'n^2 + c'' = \Theta(n^2)$

Coût du Quicksort dans le cas le plus favorable

Dans ce cas, **le partitionnement est le plus équilibré**, i.e. il produit toujours 2 tableaux de même taille (ou presque: $\sim \frac{n}{2}$)

Dans ce cas, **le partitionnement est le plus équilibré**, i.e. il produit toujours 2 tableaux de même taille (ou presque: $\sim \frac{n}{2}$)

- le partitionnement est toujours linéaire (de la forme $an + b$)

Coût du Quicksort dans le cas le plus favorable

Dans ce cas, **le partitionnement est le plus équilibré**, i.e. il produit toujours 2 tableaux de même taille (ou presque: $\sim \frac{n}{2}$)

- le partitionnement est toujours linéaire (de la forme $an + b$)
- les 2 appels récurifs sur un tableau de taille $\frac{n}{2}$ de coût $T^{\min}(\frac{n}{2})$.

D'où l'équation de récurrence

$$T^{\min}(n) = \begin{cases} c & \text{si } n = 0 \\ T^{\min}(\frac{n}{2}) + an + b & \text{sinon} \end{cases}$$

Qui a pour solution $T^{\min}(n) = c' n \log(n) + c'' = O(n \log(n))$